



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001-2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru - 560 111. India

DEPARTMENT	ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING				
Semester: VII	Course Title: Data Security & Privacy Lab			Course Code: 22AI73	
PROGRAM QUESTION	Build a Vulnerability Analyser Tool in Python that can scan source code files (any programming language) for common security vulnerabilities or bad coding patterns.				
STUDENT NAMES	Udayaraj		USN	1DS22AI057	
LAB MARKS	Program Execution (10)	Source Code (10)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	Cursor				
MAPCOURSE OUTCOME	CO1				
MAP PROGRAM OUTCOME	PO1,PO2, PO3, PO4, PO5				
MAP PROGRAM SPECIFIC OUTCOME	PSO1, PSO2				
UML / SEQUENCE DIGRAM	<pre>sequenceDiagram actor User participant Tool1 as Tool participant Rules1 as Rules participant Tool2 as Tool participant Rules2 as Rules User->>Tool1: Select & submit source code file Tool1->>Rules1: Parse source code (language-agnostic) Rules1->>Tool2: Check code against vulnerability/bad pattern rules Tool2-->>Rules2: List of detected issues (if any) Rules2-->>Tool1: Return vulnerability report Tool1-->>User:</pre>				
COURSE COORDINATOR	Dr ARUNA M G & Prof ENSTEIH SILVIA				

Code for Program 4:

4) Build a Vulnerability Analyser Tool in Python that can scan source code files (any programming language) for common security vulnerabilities or bad coding patterns.

```
import streamlit as st, tempfile, json, re
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
class Rule:
```

```
    def __init__(self, id, name, pattern, severity, fix):
```

```
        self.name = name
```

```
        self.pattern = re.compile(pattern, re.IGNORECASE | re.MULTILINE)
```

```
        self.severity, self.fix = severity, fix
```

```
    def matches(self, text): return list(self.pattern.finditer(text))
```

```
class Scanner:
```

```
    def __init__(self, rules): self.rules = rules
```

```
    def scan_file(self, path):
```

```
        try: text = path.read_text(encoding="utf-8", errors="ignore")
```

```
        except: return []
```

```
        lines = text.splitlines()
```

```
        res = []
```

```
        for r in self.rules:
```

```
            for m in r.matches(text):
```

```
                line = text.count("\n", 0, m.start()) + 1
```

```
                snippet = lines[line-1] if 0 <= line-1 < len(lines) else ""
```

```
                res.append({"severity": r.severity, "line": line, "rule_name": r.name,  
                           "snippet": snippet.strip(), "recommended_fix": r.fix})
```

```
        return res
```

```
rules = [
```

```
    Rule("R001", "Use of eval/exec", r"\b(eval|exec)\(", "HIGH", "Avoid eval/exec."),
```

```
    Rule("R002", "Hardcoded secrets", r"(api[_-]?key|password|secret)\s*=\s*[\"'].+[\"]", "CRITICAL", "Use env vars."),
```

```
    Rule("R003", "Weak hash algorithm", r"\b(md5|sha1)\(", "HIGH", "Use SHA-256, bcrypt, etc."),
```

```
]
```

```
if files:
```

```
    with tempfile.TemporaryDirectory() as tmpdir:
```

```
        for f in files:
```

```
            p = Path(tmpdir) / f.name
```

```
            p.write_bytes(f.read())
```

```
            findings += Scanner(rules).scan_file(p)
```

```
        if findings:
```

```
            st.success(f'Found {len(findings)} issues.')  
            st.dataframe(pd.DataFrame(findings))
```

```
            st.download_button("Download JSON Report", json.dumps(findings, indent=2), "findings.json", "application/json")
```

```
        else: st.info("No issues found.")
```

RESULT:

